

AI Engineering Lab Knowledge Survey — Post-Test

Form B

Why we are asking

The goal of this lab is that you understand how agentic systems work, not merely that your team ships a working harness. To find out whether the lab actually achieves that, we measure what you know about agentic systems *before* the lab starts and *again after* it ends. Comparing the two tells us where the lab teaches well and where we need to improve it. The same questionnaire structure is used at both points in time.

This is voluntary and does not affect your grade

- Participation is entirely **voluntary**. You may skip any question, or stop at any point.
- Your answers have **no influence whatsoever on your grade**. They are not part of the daily reports, the presentations, the written report, or the oral exam.
- The questionnaire is **pseudonymous** such that only you know the pseudonym: do not write your name, your FIM account, or your matriculation number anywhere on this form.
- The knowledge questions are **not an exam**. In the pre-test especially, most topics have not been taught yet, so a low score is entirely expected. Please do not guess wildly — use the “I do not know” box instead. That is more useful to us than a lucky guess.
- The full answer key will be published after the post-test, so you can learn from them.

Data protection

We process only the answers on this form, linked by the self-generated code below — no names and no identifiers that let us contact or identify you. Data are stored on university infrastructure, evaluated in aggregate, and reported only in aggregate. No result will be reported in a way that could single out an individual (groups smaller than five are not reported separately). Legal basis is your consent, which you may withdraw — though once forms are pooled we can no longer locate an individual questionnaire, so withdrawal is only possible before pooling. Questions about the survey or the data: steffen.herbold@uni-passau.de.

☐ **I have read the information above and agree to take part.**

(If you do not agree, please hand in the form blank so that we can account for it.)

Your personal code

We need to match your pre-test to your post-test, but we do not want to know who you are. Build your code from the four rules below: they give the same result on both occasions, and we cannot reverse them. One character per box, in CAPITAL letters.

Box 1 First letter of your **mother's** first name

Box 2 First letter of your **father's** first name

Boxes 3–4 **Day** of your own birthday, two digits (e.g. the 7th → 07)

Box 5 Last digit of the **house number** you grew up in

Your code:

Example: mother Anna, father Bernd, born on the 23rd, house number 14 → AB234

Part A What you did during the lab

These questions describe how you spent the three weeks. They help us understand which parts of the lab produced learning. There are no good or bad answers here, and nothing you write influences your grade — please answer honestly rather than favourably.

A1 On how many of the four working days (Mon–Thu) were you present in the lab?

This refers to in-presence, where you have access to the teaching assistants.

	0	1	2	3	4
Week 1	☐	☐	☐	☐	☐
Week 2	☐	☐	☐	☐	☐
Week 3	☐	☐	☐	☐	☐

A2 Roughly how many hours per week did you work on the lab in total

This includes your full time spent, including work outside the lab room.

☐ <10 ☐ 10–19 ☐ 20–29 ☐ 30–39 ☐ 40 or more

A3 Which components of the harness did you built

For each component of your team’s harness, tick the column that best describes your own involvement. “Reviewed only” means you read and understood a teammate’s code without writing it.

Component	I built it	co-built it	reviewed only	not involved / not built
The agentic loop (prompt–reason–act control flow)	☐	☐	☐	☐
Context assembly and management	☐	☐	☐	☐
The tool registry and built-in tools	☐	☐	☐	☐
The terminal frontend (CLI)	☐	☐	☐	☐
Containerisation of the application	☐	☐	☐	☐
MCP client integration and tool discovery	☐	☐	☐	☐
Your team’s own MCP server extension	☐	☐	☐	☐
Persistent long-term memory	☐	☐	☐	☐
Lifecycle hooks / instrumentation mechanism	☐	☐	☐	☐
Metrics, Prometheus endpoint, dashboard	☐	☐	☐	☐
Structured logs or traces of individual runs	☐	☐	☐	☐
Permission management (allow / deny / confirm)	☐	☐	☐	☐
Sub-agent integration and delegation	☐	☐	☐	☐
The automated test suite	☐	☐	☐	☐

A4 How heavily did you use AI assistance (LLMs, coding agents) for your own lab work?

Judge your own use, not your teams.

☐ not at all ☐ sometimes ☐ about half of the work ☐ often ☐ for nearly everything

A5 Of the code you submitted, how much could you explain line by line if asked right now?

Judge your own code understanding, not your teams.

☐ little of it ☐ some of it ☐ about half of it ☐ most of it ☐ all of it

A6 How much did you learn from each of the following?*Attribute how much you learned to the different source of knowledge.*

I learned a lot from ...	1	2	3	4	5
writing the code myself	☐	☐	☐	☐	☐
debugging failures in our own harness	☐	☐	☐	☐	☐
reading my teammates' code and reviewing it	☐	☐	☐	☐	☐
the weekly task sheets	☐	☐	☐	☐	☐
discussions with the teaching assistants	☐	☐	☐	☐	☐
the other teams' presentations	☐	☐	☐	☐	☐
the architecture/loop diagrams we had to draw	☐	☐	☐	☐	☐
writing the daily reports	☐	☐	☐	☐	☐
background reading I did on my own	☐	☐	☐	☐	☐

1 = strongly disagree 2 = disagree 3 = neutral 4 = agree 5 = strongly agree

Part B Your own assessment of your knowledge

This part asks what you think you know. Answer as of today.

B1 The learning objectives of this lab

Judge your current knowledge

Today, I would say ...	1	2	3	4	5
I know what the components of an agentic system are, how they work, and the role each one fulfils.	☐	☐	☐	☐	☐
I understand how these components interact with each other, and how those interactions extend what an agent can do.	☐	☐	☐	☐	☐
I understand how to ensure the quality of the individual components of an agentic system, and of the system as a whole.	☐	☐	☐	☐	☐
I understand which telemetry data is relevant for monitoring an agentic system.	☐	☐	☐	☐	☐
I could implement, monitor, and maintain an agentic system, including its source code, tests, and telemetry.	☐	☐	☐	☐	☐

1 = strongly disagree 2 = disagree 3 = neutral 4 = agree 5 = strongly agree

B2 Individual components

*For each component, give two ratings: how well you could **explain** it to a fellow student, and how well you could **implement** it yourself in code without prior research. These are deliberately separate — being able to explain something is not the same as being able to build it.*

Component	I could explain it					I could implement it				
	1	2	3	4	5	1	2	3	4	5
The role of the LLM in an agentic system	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐
The agentic loop and its control flow	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐
Context and context engineering	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐
Tools and tool interfaces	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐
The Model Context Protocol (MCP)	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐
Skills	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐
Long-term memory	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐
Hooks	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐
Permission management	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐
Sandboxing	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐
Sub-agents and agent-to-agent communication	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐
Observability and telemetry for agents	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐
Testing and quality assurance of agentic systems	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐

1 = not at all 2 = a little 3 = partly 4 = mostly 5 = fully

B3 Confidence in specific situations

Consider what you would feel comfortable doing as part of professional work.

I am confident that I could ...	1	2	3	4	5
design the architecture of an agent harness from scratch, given a set of requirements.	☐	☐	☐	☐	☐
work out why a particular agent run failed, given its logs and traces.	☐	☐	☐	☐	☐
decide what should be measured to operate an agentic system in production.	☐	☐	☐	☐	☐
judge whether someone else's agent harness is well built.	☐	☐	☐	☐	☐
explain to a non-expert why an agent needs a harness at all.	☐	☐	☐	☐	☐

1 = strongly disagree 2 = disagree 3 = neutral 4 = agree 5 = strongly agree

Part C Knowledge questions

20 questions, each with exactly one best answer. Tick one box per question. If you do not know, please tick "I do not know" rather than guessing — an honest blank is more useful to us than a lucky guess, and none of this affects your grade. Plan for about 15 minutes.

- Which statement about the LLM inside an agentic system is correct?
 - It accumulates knowledge about the current task across the calls of one run.
 - It keeps the tool results it has already seen, so they need to be sent only once.
 - It has no memory of previous calls, so the harness has to resend everything it needs.
 - It remembers the system prompt after the first call and does not require it again.

☐ I do not know.
- A team observes that their agent follows instructions reliably in short runs but starts ignoring them in long ones, although the context window is never exceeded. What is the most plausible explanation?
 - The system prompt is silently truncated once the conversation passes a certain number of messages.
 - Instructions expire after a fixed number of turns and have to be repeated.
 - The model switches to a smaller variant automatically to keep response times constant.
 - The model's effective attention to any single instruction falls as the window fills up with lower-signal material.

☐ I do not know.
- Your harness summarises the oldest third of the conversation whenever the context exceeds a threshold. What are you accepting in exchange?
 - The agent can no longer call any tool that was used in the summarised part of the run.
 - Detail from the summarised turns is no longer available in full, so some fidelity is traded for sustained coherence and lower cost.
 - The summary has to be produced by a different model from the one running the loop.
 - The loop's termination conditions stop applying, because the iteration count is reset.

☐ I do not know.

4. Your agent occasionally gets stuck calling the same failing tool over and over until you kill the process. Which property of the loop is missing?
- (a) A retry policy that repeats each failing tool call until it succeeds.
 - (b) A larger context window, so that the repeated observations still fit.
 - (c) A sandbox, so that repeated tool calls cannot damage the host system.
 - (d) A bound on execution — for example a maximum number of iterations — that forces the machine into a terminal state.
- ☐ *I do not know.*
5. Some harnesses build planning instructions into the system prompt rather than waiting for a user to ask for a plan. What follows directly from that design?
- (a) Every run spends its first reasoning effort on producing a plan, whether or not the task needs one.
 - (b) The user can no longer influence how the agent approaches the task.
 - (c) The agent's tool calls are validated against the plan by the tool registry.
 - (d) The plan replaces the conversation history as the agent's context.
- ☐ *I do not know.*
6. Which of these would be implemented as a tool rather than as some other component of the harness?
- (a) A bundle of instructions that teaches the agent how to fill in a particular PDF form.
 - (b) A callback that runs before every action to check whether it is permitted.
 - (c) A function that lists the files in a directory and returns the listing to the model.
 - (d) A policy that decides whether an action requires the user's confirmation.
- ☐ *I do not know.*
7. Your harness catches every exception raised inside a tool and returns the literal string **error** to the model. Which consequence follows most directly?
- (a) The harness can no longer log the original exception, because it has been replaced.
 - (b) The model has nothing to work with in order to choose a corrective action, so it cannot recover from failures it could otherwise handle.
 - (c) The tool's schema becomes invalid, because its declared return type is not met.
 - (d) The agent treats every failure as a final answer and ends the run.
- ☐ *I do not know.*
8. In the architecture of a harness, what role does an MCP server play?
- (a) It hosts the language model that the agent sends its requests to.
 - (b) It stores the agent's conversation history so that it survives a restart.
 - (c) It provides tools that the agent can discover and call over a standardised interface.
 - (d) It enforces which of the agent's actions require the user's approval.
- ☐ *I do not know.*

9. A team adds a new tool to their MCP server. Their agent picks it up on the next run with no code change and no configuration change. Which property of their integration does that demonstrate?
- (a) Tools are discovered at runtime, so the agent's action space follows the server's.
 - (b) The tool schemas are compiled into the harness at build time.
 - (c) The model was fine-tuned on the server's tool catalogue.
 - (d) The tool registry caches every tool it has ever seen and replays it.
- ☐ *I do not know.*
10. A harness holds a library of 60 skills, but a given run typically loads two or three of them. What mechanism makes that possible?
- (a) The skills are disclosed just in time: only an identifier and a short description are visible until one is actually needed.
 - (b) The skills are compressed and transparently decompressed by the context manager.
 - (c) The agent requests skills from the model provider's servers rather than from the harness.
 - (d) The unused skills are held in long-term memory and summarised into the context.
- ☐ *I do not know.*
11. Which requirement can be met *only* by a long-term memory component, and not by context management alone?
- (a) Older turns of the current conversation are summarised in order to save tokens.
 - (b) Information stored during one session is still available after the application has been restarted.
 - (c) The tool schemas stay visible to the model throughout the run.
 - (d) The agent's reasoning from the previous iteration informs the next one.
- ☐ *I do not know.*
12. At which points would a hook typically be triggered?
- (a) Only once, when the agent process starts up.
 - (b) Whenever the user types a new message, and at no other time.
 - (c) Whenever the context window is about to overflow, and at no other time.
 - (d) Before a tool call and after an LLM response.
- ☐ *I do not know.*
13. Your team has to record a metric for every tool call, every model call and every permission decision, without scattering counter increments throughout the codebase. Which mechanism gives you that?
- (a) A tool that the agent calls whenever it wants something to be counted.
 - (b) A wrapper around the metrics endpoint that infers the events from HTTP traffic.
 - (c) Lifecycle hooks at those points, with the instrumentation attached to the hooks.
 - (d) A sub-agent whose task is to watch the main agent and report what it did.
- ☐ *I do not know.*

14. In a model with a read-only tier, a sandbox-edit tier and a full-access tier, which action is characteristic of the sandbox-edit tier?
- (a) Writing a file inside the container's workspace.
 - (b) Sending an email on the user's behalf.
 - (c) Searching the repository for a string.
 - (d) Deleting a branch on the remote repository.
- ☐ *I do not know.*
15. Your permission system is configured so that every single tool call requires the user's confirmation. Which criticism of that configuration is valid?
- (a) It makes the permission decisions impossible to record in the observability data.
 - (b) It gives up the benefit of the tiers: routine, low-risk observation interrupts the user as often as genuinely consequential action.
 - (c) It means the sandbox is no longer needed, since the user approves everything.
 - (d) It prevents the agent from using tools discovered from an MCP server.
- ☐ *I do not know.*
16. A team argues that because their agent runs inside a container, they do not need permission management. Why is that argument wrong?
- (a) A sandbox bounds what the agent *can* reach; permission management decides which of the reachable actions it *may* take, including ones whose effects leave the sandbox.
 - (b) Containers isolate only the network and not the file system, so file access still needs a policy.
 - (c) Permission management is what starts and stops the container, so it cannot be removed.
 - (d) A container's isolation expires after a fixed time, after which a policy has to take over.
- ☐ *I do not know.*
17. Why does giving a sub-agent its own context window help the system scale?
- (a) The two agents can call the LLM simultaneously, halving the run's latency.
 - (b) The sub-agent's context does not count towards the provider's token limits.
 - (c) The main agent can then run on a smaller model, since the reasoning happens in the sub-agent.
 - (d) The coordinating agent's context stays focused, because only the summary of the sub-agent's work is added to it.
- ☐ *I do not know.*
18. A dashboard for an agent harness shows only the number of completed runs per hour. What is the most important thing it is missing?
- (a) The version number of the harness that produced them.
 - (b) The number of lines of Python in the tool registry.
 - (c) Whether those runs succeeded or failed.
 - (d) The physical location of the server that ran them.
- ☐ *I do not know.*

19. Why is it not enough to expose only aggregate metrics for an agentic system?
- (a) Aggregate metrics cannot be scraped by a Prometheus-compatible server.
 - (b) Aggregate metrics are only valid if the agent is deterministic, which it is not.
 - (c) Aggregate metrics are computed by the model itself and are therefore unreliable.
 - (d) They show that something is wrong but not why; reconstructing what happened in one run requires per-run traces or structured logs.
- ☐ *I do not know.*
20. Why should the assertions of an end-to-end test for an agent target the resulting state rather than the agent's wording?
- (a) Because the agent's messages are not available to the test harness.
 - (b) Because the same correct outcome can be reached with different phrasings and different sequences of calls.
 - (c) Because state assertions are the only kind a test framework is able to express.
 - (d) Because the wording of the agent's messages is fixed by the model provider.
- ☐ *I do not know.*

Thank you for taking part, and for three weeks of work on the lab.